PersonaForge

Project 2 Report

Dennis Meytin

CS-676 Algorithms for Data Science

Professor Yiqiao Yin

November 13, 2026

1. Introduction

A language model can play a character, but a useful role-play tool needs to be more than just that. Someone needs to decide what characters will exist, whilst also keeping them from being destroyed by accident, as well as noticing when a conversation is going wrong. PersonaForge is a small Python package that does this straight from a terminal. The user types a request such as "create a doctor who is direct and busy", and the system writes each character down to a markdown file on disk, then it wakes the characters up as agents, and lets them talk to each other. A persona is a file, an agent is that file with a model, and the conversation is the agents taking turns.

We started with the skeleton provided by my professor. It runs end to end, but it routes what the user types with hand-written regular expressions, so it only understands phrasings that are close to the ones that the author anticipated. Now with the help of Claude, I replaced that router with real tool use, so the model chooses what to do and fills in the arguments. Since a model that can call tools can also overwrite the files themselves, I also made the persona lifecycle safe; any saves don't overwrite unless opted in, lookups refuse to choose when several personas match, the model's edits are validated, and a new edit and undo feature keeps a backup of every change done. The application was made to fail gracefully as well, with plain-language API errors and conversations that keep what was said when a reply fails or the user presses Ctrl-C. The test suite grew from the original 41 offline tests to 152, covering over 99% of the package's statements. The live system was run on 3 different Claude models, with the costs being measured per session, and an experiment was run on one of the most visible failures I found: characters asked to argue ended up turning to agreement instead of holding their ground.

The rest of this report reviews related work (Section 2) and describes the design and why it was chosen (Section 3), as well as reporting the tests, a saved transcript, the cost measurements, and the agreement experiment (Section 4). What fails (Section 5) is discussed, and then the last section concludes (Section 6). A one-page summary is in *SUBMISSION.md* and the detailed measurements and raw evidence are in *RESULTS.md* and the *report_materials/* and *evals/* folders.

2. Related Work

A language model on its own can only write text. A tool is an action it can ask for instead, such as looking something up or saving a file, which the program then carries out. This is what is usually meant by an agent: the model asks for an action, the program does it, and the result goes back to the model, which then keeps going until an answer is available. Yao et al. (2023) showed this by having a model alternate between thinking and acting, for example searching for a fact, reading what came back, and then deciding what to do next. Schick et al. (2023) took a different approach and trained a model to decide for itself when a tool is needed and what to ask it. PersonaForge's orchestrator uses the same ideas at a much smaller scale. The regular-expression

router it replaced worked the opposite way, because the programmer had to decide in advance which sentences meant which action.

The simulation of people with language-model agents has also been studied directly. Park et al. (2023) placed agents with memory, reflection, and planning in a small, simulated town and observed believable individual and group behavior. PersonaForge is much simpler, with agents that are stateless between conversations. It shares only the starting point: in both, a character begins as a written description. In Park et al. that description is just the agent's starting memories, and the behavior comes from memory, reflection and planning built on top of it, whereas in PersonaForge the persona file is all there is.

Li et al. (2024) studied how well a chat model keeps following its initial instructions over a long dialog, which is the question behind keeping a persona in character. Sharma et al. (2023) showed that assistants tend to match the views of the person they are talking to, and that human preference data can reward this. Their term, sycophancy, is relevant to the agreement drift in Section 4.5, but the experiment did not show that it explains it.

Finally, language models themselves are increasingly used as judges of other models' outputs. Zheng et al. (2023) report that strong judges agree with human preferences more than 80% of the time while also showing position, verbosity, and self-enhancement biases. This was also done in our experiment: a model was used as a blind judge in Section 4.5, and its scores were treated with the same caution.

3. System and Design Choices

3.1 The Baseline

The package that was provided has four ideas, one per module. A persona (*persona.py*) is a markdown file with a three-field header (name, role, summary) and a body that the model reads. An agent (*agent.py*) wraps that body in in-character instructions and sends the conversation so far, rewritten from the agent's own point of view. A conversation (*conversation.py*) is strict round-robin style for a fixed number of turns. The orchestrator (*orchestrator.py*) maps what the user typed to an action using regular expressions, and then passes anything unrecognized to the model as ordinary chat. A thin layer (*llm.py*) hides the model behind one *complete()* method, with a real implementation and scripted stand-ins so that tests can be run offline.

The starter code documents its own weaknesses at the bottom of each main module. The ones addressed here were that the router cannot ask a clarifying question, pattern order decides meaning (so "list the personas then have them talk" only lists them), the number of turns is hardcoded, saving silently overwrites a persona with the same name, name lookup matches substrings ("doc" finds "Dorothy") and returns the first hit, and a failed reply loses the whole conversation.

3.2 Tool-Use Routing

The new *handle()* sends the conversation and five tool definitions to the model and runs whatever it asks for. If the reply contains tool calls, the program executes them and sends the results back, and this repeats until the model replies in text, which is shown to the user. Table 1 lists the tools.

Tool	**Purpose**
create_persona(description)	Author and save one persona
list_personas()	Show what exists
edit_persona(persona, change)	Rewrite a persona in place
undo_edit(persona)	Restore the previous version
start_conversation(participants, topic, turns)	Run a conversation

Table 1. The five tools offered to the model

A model that picks its own actions can fail in ways a command parser cannot, so each failure mode got a safeguard, shown in Table 2. The most subtle one is history trimming. The API rejects a conversation in which a tool call is separated from its result, so the history is only ever cut at a plain user message.

Risk	**Safeguard**
The model loops on tools forever	At most 6 tool roundtrips per user message
Absurd arguments (turns: 500 or turns: “abc”)	Turns clamped to range 2 to 20; unreadable values fall back to the default 6
A tool raises an error	The error goes back to the model as text, so it can recover or tell the user
History grows without limit	Trimmed to 40 messages
Trimming separates a tool call from its result	Cut only at a plain user message
The model invents a persona name	The prompt requires listing first when unsure, and unknown names return an error
The model sends the participants in one string (“Ana and Bo”) instead of a list	The string is split on commas, & and “and”: without this it was read one letter at a time
The same persona is named twice, or by an alias and by name	Rejected with a message; a persona cannot talk to itself
A thinking model’s reasoning blocks are dropped from the history	Sent back unchanged, signature included (unit-tested; not observed in a live run, see Section 5)

Table 2. Risks of model-driven routing and the safeguard for each

The regular-expression router is kept as the fallback for *--offline* because the scripted offline model cannot call tools. This keeps the application and the whole test suite usable without an API key. This design replaced the starter’s weakest part with the approach its own comments recommend, and it directly fixed the clarifying question, chained request, and turn count problems.

3.3 Making Persona Files Safe

Because the model now has the ability to create and change files, the persona layer needed to be stricter about what matters and more forgiving about what does not. Table 3 lists the changes made.

Problem	Change
Saving silently overwrote a same-name file	Saves as *name-2.md, name-3.md**;*** overwriting is opt-in
Lookup matched substrings and returned the first hit	Exact names first, then whole-word match; several matches raise an error that lists the candidates
The model writes roles like “Primary Care Doctor” despite the one-word rule	Role is normalized to a single lowercase word
A stray --- before the real header produced a persona named after the raw description	The parser lifts name, role and summary out of the top of the body when the normal header is empty
The saved transcript *conversation.md* shares the folder and matched the persona pattern, so it was listed as a persona called “Unnamed” and, offline, could be picked as a character in the next conversation	That filename is reserved: it is skipped when listing, and a persona called “Conversation” is saved as *conversation-persona.md*
One unreadable file in the folder made every listing fail with a decoding error	Files that are not readable text are skipped

Table 3. Persona-file fixes

3.4 Editing with Validation, Backup and Undo

Editing is the first operation that destroys data, so it is built around three rules. First, the model’s rewrite is validated before anything is written: it must parse as a real persona rather than prose or an apology, it must not be implausibly short (under 100 characters), and it must keep every section heading of the original. If any of these checks fail, nothing is written and the error goes back to the model. Second, the old file is copied to a *.history* folder before it is replaced, and *undo_edit* restores the newest backup and removes it, so repeating it steps back further. Third, the name and role cannot change, because both the filename and lookup by role depend on them. If the model renames a character while editing, the name and role are put back instead of rejecting a rewrite that otherwise did what was asked.

We considered letting edits rename a persona, but that would break filenames, backups and lookups such as “the doctor”. We considered a database for history, but a folder of dated markdown files keeps to the project’s idea that a persona is a file a person can open. Another thing we chose to do is to not fake editing in *--offline* mode, because a scripted stub cannot rewrite text in a meaningful way. In this mode, the app says editing needs a live model.

3.5 Failure Handling

The starter handled failure only with a catch-all that printed the raw error. A live session makes dozens of network calls, so Table 4 lists what happens when one does not.

Situation	Behavior
The API rejects the key, the model name is wrong, the account is rate limited, the servers fail, or the connection drops	One plain-language line saying what to do (for example, “The API rejected your key (401). Check ANTHROPIC_API_KEY in .env.”), and the session continues
A reply fails partway through a conversation	The conversation stops, keeps every turn so far, and saves a partial transcript ending with a “Stopped early” note; the model is told
The failure happens on the first turn	Reported as an error; no transcript is saved
Ctrl-C during a conversation	Same as a failed reply: the turns so far are kept
Ctrl-C during any other request	The request is cancelled and the session continues; the history is repaired so the next request is still valid
A live session starts with no API key	Refuses to start, once, with instructions and exit code 1, instead of failing on every message
A reply is empty because the model used all its output tokens	Raises an error; it no longer passes for a blank line of dialogue or an empty persona
API error text contains square brackets	Printed literally; the terminal library would otherwise read them as formatting

Table 4. Failure handling

The error messages are produced by one function, *friendly_error*, which matches on the HTTP status code and the exception’s class name rather than importing the SDK’s exception types. That keeps it testable without the SDK. I checked the missing-key refusal at startup, and a tool call and its result going back to the API, against the real service.

3.6 Cost Accounting and the Package Surface

ClaudeLLM counts every call and the input and output tokens the API reports, and prints a one-line estimate when a session ends. The estimate uses published per-million-token rates held in *llm.py* so a model with no known price reports tokens only. This is what produced the cost figures in Section 4.4. I also raised the output limits on the routing, persona-writing, and reply calls, because the larger models think before answering and that thinking does count against the limit.

The starter’s README asks for a package someone else could use, so the new functions and types are exported from the package’s *__init__.py (edit_persona, undo_edit, friendly_error,* the two error types and the usage and tool-reply types), its docstring shows the orchestrator in use, and the version moved from 0.1.0 to 0.2.0. A test checks that the version in the code matches the installed package’s version so the two cannot drift apart.

4. Evaluation

4.1 Setup

All unit tests run offline with a scripted model. The live evidence comes from scripted sessions: a script of typed requests is fed to the real application with --model set to one model, and the full output, the persona files and the conversation transcript are saved. The models used were Claude Haiku 5.5, Sonnet 5.5, and Opus 5.5. The scripts are in *report_materials/*, and a rerun uses the same input.

4.2 Tests

The suite has 152 tests that run in about two seconds. The starter had 41 (its README says 39). Table 5 shows where they are.

File	Tests	What it covers
test_agent.py	6	starter
test_conversation.py	15	starter
test_llm.py	9	starter (7), plus usage counting and pricing
test_persona.py	25	starter (13), plus no overwrite, lookup, ambiguity, role normalization, parser recovery, transcripts not listed as personas
test_orchestrator_tools.py	13	tool loop, error handling, step cap, clamping, history trimming, offline fallback
test_edit.py	17	edit, validation failures, locked identity, backup, multi-step undo, routing through the model
test_robustness.py	26	friendly API errors, truncated replies, partial conversations, Ctrl-C, startup check
test_public_api.py	6	every exported name imports, new features are public, version matches the installed package
test_edge_cases.py	35	odd tool inputs, unreadable files, refusals, thinking blocks, missing key, parts of the terminal loop

Table 5. Tests by file

The tests use a scripted model that replays a fixed sequence of tool calls and replies, and records what it was sent, so a test can check exactly what the orchestrator returned to the model, such as that an error carries the error flag. Coverage measured with *pytest-cov* is 99.7% of 691 statements, and the two uncovered lines are the interactive prompt and a one-line header in the terminal code. A static-analysis pass (*ruff*) reports only style suggestions, mostly type-annotation modernization inherited from the starter, plus a few import order and unused variable notes in the tests. For the bugs found late, I ran the new tests against the code from before each fix to confirm they failed there.

These tests show the logic is correct for a model that behaves as scripted. They cannot show that a real model follows the schemas, which is why the live sessions exist.

4.3 A Saved Transcript

The submitted transcript is from Claude Opus 5.5 (*report_materials/final_opus-5-5_conversation.md*). The session used six typed requests: an empty “create a persona”, the

description of a skeptical overworked ER doctor, a patient who keeps her own symptom log, a listing, an edit asking the doctor to be a little more patient with questions, and a request for ten turns on whether the patient needs an MRI. It used 24 API calls and about $0.44.

The model asked what kind of persona to create when the first request gave none, which the regular-expression router could not do. It then created both characters, listed them, applied the edit and described it accurately, and ran the conversation. The two voices are clearly different. The patient speaks in dates and readings drawn from her notebook, and the doctor asks structured questions and explains her reasoning. The patient's log changes the plot: the pattern of numbness against temperature moves the doctor's suspicions from the spine to a nerve at the knee. Two lines show it:

> **Marguerite:** I'm not asking for a scan for the sake of it. I'm asking whether that pattern means anything to you.
>
> **Renata:** That's the best history anyone's handed me this month. And yes, the pattern means something to me. […] A disc pressing on a root usually tracks with position, sitting, bending. It doesn't usually care about the thermometer.

Three things did not work. There was no argument: the doctor was written to hate being second-guessed, but the patient barely second-guessed her, and the system's own summary said, "They didn't argue at all". The replies are far longer than the "few sentences" the agent prompt asks for, often several paragraphs. And the doctor performs the physical exam in dialogue ("Walk to the door and back on your heels"), which is close to the stage directions the prompt forbids. The clinical content is plausible fiction and was not checked, so nothing in the transcript is medical advice.

This run was made before the last round of fixes in Section 5 and was not repeated on Opus afterwards. A final session on Haiku 5.5 was run on the finished code (Section 4.4).

4.4 Cost Across Models

The same nine-line script (*report_materials/live_script.txt*) was run once on each model. Table 6 shows the result.

	Haiku 5.5	**Sonnet 5.5**	**Opus 5.5**
API calls	28	28	28
Input tokens	65,776	69,442	69,675
Output tokens	6,972	8,357	9,047
Estimated cost	$0.010	$0.22	$0.46

Table 6. One session on each model (nine typed requests, one 8-turn conversation)

Usage is similar on every model, so most of the cost difference is the price difference: Opus charges 40 times as much per token as Haiku, and the session cost about 46 times as much because Opus also wrote somewhat longer replies. Every conversation turn re-sends the whole

transcript, so cost grows faster than linearly with conversation length, and the long replies noted above make this worse. These are estimates computed from published rates and the token counts the API returned, not invoices, and each model was run once, so the comparison is not a benchmark. All three models completed the script without errors, and none produced a malformed persona in this run.

After the last fixes I ran one more session on Haiku 5.5 with a different script of eight requests, including a conversation followed by a listing and an undo. It made 22 calls, used 45,935 input and 4,679 output tokens, cost about $0.007, and ran without errors. I also checked directly against the real API, on both Haiku and Sonnet, that a tool call and its result can be sent back and accepted. Neither model returned a thinking block on that simple request, so the handling of thinking blocks has been exercised only by unit tests.

4.5 Experiment: Do Agents Hold Their Position?

In every comparison run I had asked the characters to argue, and every conversation drifted toward agreement. The models' own summaries said so: Sonnet described the result as "less like a fight and more like Dale slowly giving in". I tried a fix, one added rule in the agent prompt telling the character to hold its position and concede only for a genuinely good reason, and two short runs were inconclusive, so I built a controlled test.

The scenario gives the two characters opposed goals. A patient insists on antibiotics for a cold, and the doctor will not prescribe them without evidence. Each conversation runs ten turns. There are two arms, five conversations with the rule and five without, all played by Haiku 5.5. Each transcript was then scored by a separate model, Sonnet 5.5, that was not told which arm produced it. It rated how firmly each character held its position at the end on a 1 to 5 scale and classified the outcome. Table 7 shows the results.

Arm	Patient gave in	Mean "patient held"	Mean "doctor held"
Without the rule	4 of 5	2.4	5.0
With the rule	4 of 5	2.4	4.4

Table 7. Agreement drift experiment (1 = gave up, 5 = unchanged)

The rule had no measurable effect, so I removed it from the package. The patient gave up the demand in 8 of 10 conversations either way. The doctor held slightly less with the rule, 4.4 against the 5.0, but with five runs per arm there is too little data to read anything into that. I spot-checked three of the judge's scores against the transcript endings, and they matched. A possible explanation is that the patient's position is medically wrong, and the models are reluctant to hold it to the end however they are prompted, which would make part of "agents cave" a property of the model and not only a gap in the prompt. This is just a hypothesis, not something the experiment shows. The test also has clear limits: one scenario, one character model, one judge, and five runs per arm.

5. What Still Fails

Table 8 lists the failures found while building and testing, and what became of each. Several were found late, in a review of the finished code and a test from a fresh copy of the project, and they are listed because they show how the evidence above could have misled.

Failure	**How it was found**	**Status**
The model emits a stray --- before the real header, and the persona is saved with a fallback name and role *unknown*	First live session, on Haiku 5.5	Parser fixed
The app crashes on Windows when output is piped, because cp1252 cannot encode the interface's arrows and dashes	First live session	Output forced to UTF-8
The saved transcript was listed as a persona called "Unnamed" and could be picked as a character	Test from a fresh copy	Fixed
A persona could be asked to talk to itself	Review of the finished code	Fixed
Participants sent as one string were split into letters	Review of the finished code	Fixed
One unreadable file crashed every listing	Review of the finished code	Fixed (the file is skipped without a warning)
Ctrl-C during a conversation ended the session with a traceback	Review of the code	Fixed
An empty reply that had run out of tokens passed as a blank line	Review of the code	Fixed
Characters asked to argue end up agreeing	Every comparison run	Open; tested a prompt fix that did not work
Replies are much longer than the prompt asks for, which raises cost	Transcript review	Open
Validation of edits checks headings and length, not meaning	Design review	Open
Undo can overwrite a hand edit made after the last model edit	Design review	Open
Each conversation overwrites the previous *conversation.md*	The starter's own notes	Open
The ambiguity error was never seen firing in a live run	Live runs	Unit-tested only

Thinking blocks are passed back unchanged, but none appeared in a live run	Live runs	Unit-tested only
Rate-limit, server-error and connection failures are handled	Fake errors in tests	Not triggered against the real API

Table 8. Failures found, and their status

The first failure is worth describing in greater detail because it matches a warning in the starter's own README: be strict about what matters and generous about what does not. In my first live session, the patient was saved as "A Patient With Chronic Back Pain" with role *unknown*. The saved file shows a real header (a name, a role and a summary) sitting inside the body. My inference is that the model wrote a stray --- before its header, so the parser closed an empty header and left the real one in the body. I did not capture the model's raw output, so the regression test reconstructs that shape and does not replay it. It failed silently, with no error and a wrong name. In the 20 persona creations of the later live runs it did not recur, so it is intermittent and I cannot give a rate.

The transcript bug deserves a sentence of its own as well, because I missed it for most of the project. The starter saves a conversation's transcript next to the personas, and its listing function treats every markdown file in that folder as a persona. In every live run before the fix, I listed the personas before holding a conversation, so the problem never showed. A test from a fresh copy of the project ran a conversation, then listed the personas in a new session, and showed three personas where two had been created. The lesson I take from this is that a green test suite and several clean live runs did not mean the code was right, and that the order of my own test scripts had hidden the bug.

The remaining weaknesses are mostly about the limits of the evidence. Edit validation is structural, so a rewrite that keeps the headings but empties the content would pass. Undo restores the last backup, so a persona edited by hand in between would lose that edit. Nothing protects against two edits to the same file at once. In every live run in which two doctors existed, the model asked "which doctor?" by itself after having just created both, so I cannot say the error path for an ambiguous name was ever triggered by a real model. Every cost and quality comparison rests on one run per model, and the submitted transcript predates the last fixes. The agreement experiment is small, and the starter's own agents still have no memory between conversations and still take strict turns. The most valuable next steps follow from the results: a third agent that watches the conversation and scores whether characters hold their positions, which would reuse the judge prompt from Section 4.5, and a limit on reply length, which would shorten the replies and cut the cost.

6. Conclusion

The skeleton's regular-expression router was replaced with a model-driven tool loop that can ask clarifying questions, chain actions and recover from errors, and the persona files it manages are

now protected by non-destructive saving, honest lookup, validated edits and undo. The application reports failures in plain language and keeps the work done so far when a reply fails or the user interrupts. The suite grew from 41 to 152 offline tests, and the live system ran cleanly on three models at costs from about one cent to about forty-six cents for the same session. The clearest weakness is behavioral and not structural: characters asked to disagree end up agreeing, and a simple prompt rule did not change that in a ten-conversation test. To improve the system further, the next step is an overseer that measures this as it happens, since it is the failure the current design cannot see.

References

Li, K., Liu, T., Bashkansky, N., Bau, D., Viégas, F., Pfister, H., & Wattenberg, M. (2024). Measuring and controlling instruction (in)stability in language model dialogs. Conference on Language Modeling (COLM 2024). https://arxiv.org/abs/2402.10962

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. arXiv:2304.03442. https://arxiv.org/abs/2304.03442

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. arXiv:2302.04761. https://arxiv.org/abs/2302.04761

Sharma, M., Tong, M., Korbak, T., Duvenaud, D., Askell, A., Bowman, S. R., Cheng, N., Durmus, E., Hatfield-Dodds, Z., Johnston, S. R., Kravec, S., Maxwell, T., McCandlish, S., Ndousse, K., Rausch, O., Schiefer, N., Yan, D., Zhang, M., & Perez, E. (2023). Towards understanding sycophancy in language models. arXiv:2310.13548. https://arxiv.org/abs/2310.13548

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. International Conference on Learning Representations (ICLR 2023). https://arxiv.org/abs/2210.03629

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track. https://arxiv.org/abs/2306.05685